

DGEN TECHNOLOGIES PVT. LTD.
Kolkata, India

ADAM

Autonomous Desktop AI Module

HARDWARE & SOFTWARE SYSTEM DOCUMENTATION

v32 — Wired ESP32-CAM Architecture, Pico-Driven TFT Face, Modular Laptop Control

Consolidated Engineering Reference
Version 2.1 | 25 July 2026

i SCOPE OF THIS REVISION

This document supersedes the original ADAM v32 Master Blueprint's pinout tables. It reflects the current, working codebase: `esp32cam_adam_wired_fixed.ino`, `ADAM_Pi.py`, `adam_tft.py`, and `laptop_agent.py` — including the GPIO16/PSRAM conflict fix, the confirmed Pico-driven TFT data path, and the modular laptop-control action registry.

Table of Contents

1. System Overview & What Changed Since the Original Blueprint
2. Physical Architecture — Body & Head Zones
3. Raspberry Pi Zero 2W — Complete Pinout & Wiring
4. Raspberry Pi GPIO Reference Diagram (Official)
5. ESP32-CAM — Wired UART Architecture & Pin Map
6. The GPIO16/PSRAM Bug — Root Cause & Fix
7. TFT Display — Rendered on the Pico, Driven via the ESP32-CAM Relay
8. Power Distribution & Budgeting
9. Wire Protocol — Pi ↔ ESP32-CAM
10. Software Architecture — ADAM_Pi.py
11. Laptop Control Subsystem — laptop_agent.py
12. Serial Port Setup & Common Failure Modes
13. Setup Checklists

1. System Overview & What Changed Since the Original Blueprint

ADAM v32 is a physical desk-robot AI companion built around a Raspberry Pi Zero 2W as the primary compute unit, an ESP32-CAM handling vision and touch I/O, and a live Gemini API voice session. The architecture has evolved significantly since the original Master Blueprint PDF. This document reflects the code as it currently exists across the four core project files.

1.1 Key Architectural Changes

Area	Original Blueprint (v32 draft)	Current Implementation
Face rendering	Rendered on a Raspberry Pi Pico, touch routed via a PCF8574T I2C expander into the ESP32-CAM, emotion relayed from the Pi through the ESP32-CAM	Still rendered on the Pico (unchanged role) — but touch pads now wire directly to ESP32-CAM GPIOs, removing the PCF8574T from the touch path. <code>adam_tft.py</code> exists as a Pi-side Python port of the same face-drawing logic, for headless preview/testing off the physical Pico — it is not the live display driver on assembled units.
Touch sensors	4× TTP223 pads routed through a PCF8574T I2C expander into the ESP32-CAM	4× TTP223 pads wired directly to ESP32-CAM GPIOs — no I2C expander needed
Pi ↔ ESP32-CAM link	UART2 planned on GPIO13/GPIO15 with touch3 on GPIO15 (pin conflict, unresolved)	UART2 fixed to GPIO13 (RX) / GPIO15 (TX) on the ESP32-CAM side; TOUCH3 relocated to GPIO16 to resolve the conflict
Camera reliability	GPIO16 shared between UART2 and PSRAM chip-select — silent frame corruption	UART2 moved fully off GPIO16/GPIO4/GPIO2; PSRAM access is no longer disturbed
Laptop control	Not present in original blueprint	New subsystem: <code>laptop_agent.py</code> (runs on the user's laptop) + mDNS auto-discovery on the Pi side, self-describing <code>/actions</code> manifest feeds the Gemini tool schema automatically
Emotion routing	Pi → Wi-Fi → ESP32-CAM → wire → Pico	Pi → UART2 (text line "EMO:xxx\n") → ESP32-CAM → UART1 relay wire → Pico → TFT. Confirmed live data path for the assembled robot.
Tilt servo	ESP32-CAM GPIO13	Moved to ESP32-CAM GPIO4 (GPIO13 now carries UART2 RX)

i CONFIRMED DATA FLOW — READ THIS FIRST

CONFIRMED DATA FLOW for the assembled ADAM v32 unit:

- Camera images: ESP32-CAM → UART2 → Pi Zero 2W
- Touch state: ESP32-CAM → UART2 → Pi Zero 2W
- Emotion/face: Pi Zero 2W → UART2 → ESP32-CAM → UART1 relay → Pico → TFT

The Pico renders the face — it is not driven directly by the Pi. `adam_tft.py`'s drawing routines run on the Pi only for headless/off-hardware preview and design work, not as the live display.

2. Physical Architecture — Body & Head Zones

ADAM is split into two physical/electrical zones to minimize the number of wires crossing the mechanical neck joint and to keep high-speed camera DMA traffic isolated from audio DMA traffic.

2.1 Body (Base) — Raspberry Pi Zero 2W

- Raspberry Pi Zero 2W — main brain. Runs the Gemini Live session, audio I/O, servo pan control, memory, and the ESP32-CAM UART link.
- INMP441 I2S microphones (×2, stereo capture)
- MAX98357A I2S amplifier + 3W speaker
- RCWL-0516 microwave radar — presence detection to gate session open/close
- MG90S pan servo — left/right neck rotation, driven by gpiozero AngularServo
- 5V buck converter — steps down battery voltage; do not power peripherals from the Pi's own 5V pin

2.2 Head (Moving Part) — ESP32-CAM + Pico

- ESP32-CAM (AI-Thinker module) — camera capture, touch-pad reading, gesture detection, tilt servo control, and emotion relay to the Pico
- 4× TTP223 direct-GPIO touch pads — left cheek, right cheek, stop/petting-A, petting-B
- MG90S tilt servo — up/down head motion
- Raspberry Pi Pico — dedicated face-rendering microcontroller. Receives emotion commands from the ESP32-CAM over a local UART wire and drives the ILI9341 TFT directly (see Section 7)
- ILI9341 320×240 TFT — face display, wired to and driven by the Pico, not the Pi

2.3 Communication Flow (Flow 2 — Fully Wired, Zero Wi-Fi for the vision link)

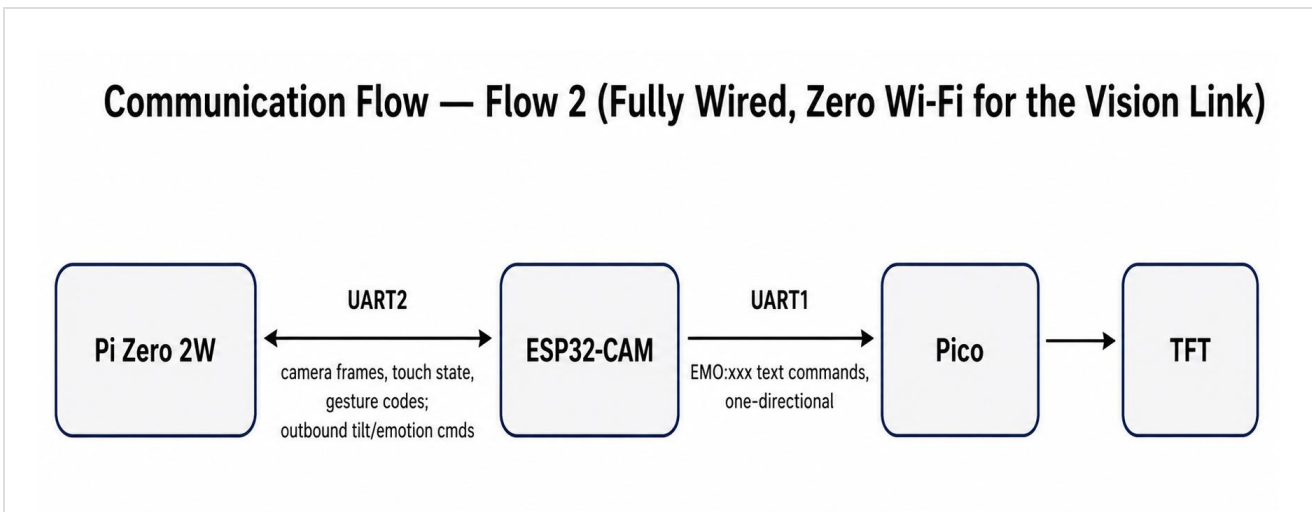


Figure 2.1 — End-to-end wired communication path: Pi Zero 2W → ESP32-CAM → Pico → TFT

Confirmed data flow for the assembled unit:

- Camera images — ESP32-CAM captures JPEG frames and sends them to the Pi Zero 2W over UART2
- Touch state — the ESP32-CAM reads its 4 touch pads and sends both raw touch bytes and decoded gesture codes to the Pi Zero 2W over UART2
- Emotion / face — the Pi decides an emotion, sends a text command down the same UART2 link to the ESP32-CAM, which relays it verbatim over its dedicated UART1 wire to the Pico; the Pico renders the corresponding face on the TFT

Only 4 wires cross the neck joint: 5V, GND, and the two UART2 data lines. The Pico and TFT live entirely inside the head, alongside the ESP32-CAM.

3. Raspberry Pi Zero 2W — Complete Pinout & Wiring

This table consolidates every GPIO the Pi actually uses across audio, servo, radar, UART, and TFT — cross-referenced against ADAM_Pi.py and adam_tft.py.

3.1 Audio (I2S) — proven working, do not modify without re-testing on hardware

Physical Pin	GPIO	Function	Connects To
12	GPIO 18	I2S BCLK (shared clock)	INMP441 mic SCK + MAX98357A amp BCLK
35	GPIO 19	I2S LRC/WS (shared)	INMP441 mic WS + MAX98357A amp LRC
38	GPIO 20	I2S Data IN	INMP441 mic SD
40	GPIO 21	I2S Data OUT	MAX98357A amp DIN

3.2 Servo and Radar

Physical Pin	GPIO	Function	Connects To
32	GPIO 12	PWM servo signal	Pan servo (orange/yellow signal wire) — NECK_GPIO_PIN in ADAM_Pi.py
36	GPIO 16	Digital input	RCWL-0516 radar OUT (presence detection)

i TFT IS PICO-DRIVEN, NOT PI-DRIVEN

The Pi Zero 2W does NOT drive the TFT in the assembled robot — the ILI9341 is wired to and rendered by the Pico in the head. See Section 7 for the Pico's own SPI pinout.

3.3 UART2 — Wired Link to ESP32-CAM (Flow 2)

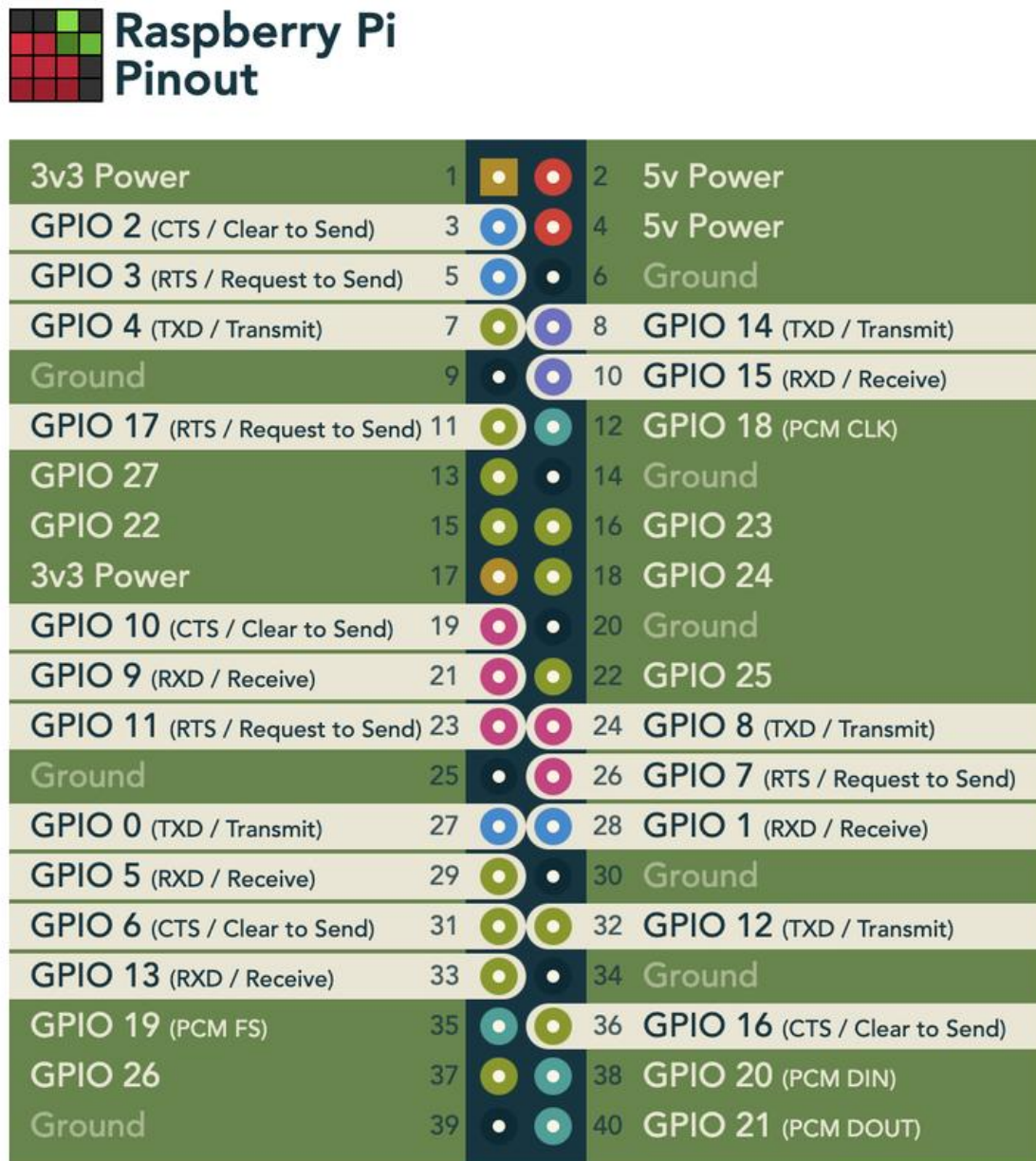
Physical Pin	GPIO	Direction	Connects To
8	GPIO 14 (TXD)	Pi transmits	ESP32-CAM GPIO 13 (UART2 RX)
10	GPIO 15 (RXD)	Pi receives	ESP32-CAM GPIO 15 (UART2 TX)

i UART CONFIG

Default port in ADAM_Pi.py is /dev/serial0 at 921600 baud (PI_UART_PORT / PI_UART_BAUD env vars). This maps to the physical UART pins above once the Bluetooth UART is disabled and serial-getty is turned off — see Section 11.

4. Raspberry Pi GPIO Reference Diagram (Official)

Use this diagram alongside Section 3 when wiring the Pi Zero 2W. Physical pin numbers (right-hand column of numbers) match the tables above; GPIO/BCM numbers are what the code (gpiozero, ADAM_Pi.py) references.



5. ESP32-CAM — Wired UART Architecture & Pin Map

Source: esp32cam_adam_wired_fixed.ino. This revision restores and corrects the emotion-relay path and, critically, resolves a PSRAM/UART pin conflict that was the root cause of ADAM's camera failing intermittently ('ADAM can't see') while touch and gestures kept working.

5.1 Final Pin Map (Corrected)

Signal	ESP32-CAM GPIO	Notes
UART2 TX (to Pi RX)	GPIO 15	→ Pi GPIO15 physical pin 10. Genuinely free — not PSRAM, not flash-strap.
UART2 RX (from Pi TX)	GPIO 13	→ Pi GPIO14 physical pin 8. Previously the tilt servo pin — servo moved to GPIO4.
UART1 TX → Pico relay	GPIO 3 (U0RXD, repurposed)	One-directional, post-assembly only. Disconnect when reflashing over USB-serial.
Tilt servo	GPIO 4	Moved off GPIO13 to free it for UART2 RX. Flash LED flicker is cosmetic only.
Touch 1 — left cheek	GPIO 12	Strapping pin — see caution below.
Touch 2 — right cheek	GPIO 14	Distinct chip from the Pi's own GPIO14 — no electrical conflict, naming coincidence only.
Touch 3 — stop / petting-A	GPIO 16	Moved here from GPIO15 to resolve the UART2/touch conflict. Safe because UART2 no longer touches GPIO16.
Touch 4 — petting-B	GPIO 2	—

5.2 Camera Bus (Standard AI-Thinker, Unchanged)

Signal	GPIO
PWDN	32
XCLK	0
SIOD (SDA)	26
SIOC (SCL)	27
Y9–Y2 (data bus)	35, 34, 39, 36, 21, 19, 18, 5
VSYNC	25
HREF	23
PCLK	22

⚠ STRAPPING PIN CAUTION

GPIO12 (TOUCH1) is a strapping pin — it affects flash voltage mode at boot. A TTP223 pad idling LOW is normally fine, but a pad that idles HIGH or floats at power-on can cause intermittent boot failures. If you see this, swap TOUCH1 and TOUCH3 (GPIO12 ↔ GPIO16), or add a pull-down resistor on GPIO12.

5.3 Wiring Checklist

- ESP32-CAM GPIO15 → Pi GPIO15 (physical pin 10) — TX→RX
- ESP32-CAM GPIO13 → Pi GPIO14 (physical pin 8) — RX→TX (double-check the TX/RX crossover — this is the #1 wired-UART mistake)
- ESP32-CAM GPIO3 → Pico UART RX (only if the Pico display path is retained) — disconnect before reflashing
- TOUCH1→GPIO12, TOUCH2→GPIO14, TOUCH3→GPIO16, TOUCH4→GPIO2
- Tilt servo signal → GPIO4
- UART2_BAUD (ino) must match PI_UART_BAUD (ADAM_Pi.py) — default 921600; drop to 460800 if frames arrive corrupted
- After flashing, open Serial Monitor at 115200 baud via the USB debug UART (GPIO1/3) and confirm "Camera ready" prints — disconnect the Pico relay wire first if wired to GPIO3

6. The GPIO16/PSRAM Bug — Root Cause & Fix

This was the actual root cause behind ADAM's vision failing while everything else (touch, gestures) kept working — worth documenting in detail since it is exactly the kind of bug that reappears if pins are ever reassigned casually.

6.1 Root Cause

The previous pin plan put UART2 RX on GPIO16. GPIO16 is the PSRAM chip-select pin on virtually every real AI-Thinker ESP32-CAM board with PSRAM populated (the standard configuration). The camera sketch uses `FRAMESIZE_VGA` with `fb_count=2`, which requires PSRAM for its frame buffers. Sharing GPIO16 with UART2 silently corrupted PSRAM access — frames came out corrupted or `esp_camera_init()` failed outright, while touch and gesture code (which doesn't touch PSRAM) kept working fine. That combination — "ADAM looks blind but nothing else seems broken" — is exactly what a GPIO16/PSRAM conflict produces.

6.2 Fix Applied

- UART2 no longer touches GPIO16, GPIO4, or GPIO2 at all — moved to GPIO13/GPIO15, which are genuinely spare, non-PSRAM, non-flash-LED pins
- Tilt servo relocated from GPIO13 to GPIO2, then finally to GPIO4 once TOUCH4 claimed GPIO2 (see final map in Section 5.1)
- All 4 touch pads now have dedicated pins with zero sharing/fallback logic — the old `PSRAM_SAFE_BOARD` and `NO_TOUCH4` fallback modes are gone
- Firmware detects PSRAM at boot (`psramFound()`) and falls back gracefully to `fb_count=1` / DRAM if it's genuinely absent, rather than silently misbehaving

6.3 Verifying the Fix on Your Hardware

Board revisions vary — verify the true PSRAM pin with a multimeter or datasheet before trusting silkscreen labels. After flashing the corrected sketch, the serial monitor should print one of:

```
PSRAM found - using 2 frame buffers in PSRAM Camera ready
```

If you instead see "Camera init FAILED", the PSRAM conflict is not fully resolved for your specific board revision — recheck which physical GPIO is actually PSRAM CS on your unit.

7. TFT Display — Rendered on the Pico, Driven via the ESP32-CAM Relay

The ILI9341 2.4" face display lives in the head and is rendered by a dedicated Raspberry Pi Pico — this role is unchanged from the original blueprint. The Pico is powered from the shared 5V rail (into VSYS or VBUS) and in turn powers the TFT panel from its own 3V3 OUT pin. It receives emotion commands over a local one-directional UART wire from the ESP32-CAM (see Section 9) and owns 100% of the drawing/animation logic.

7.1 Display Module — 2.4" ILI9341 FPC Ribbon Pinout

The panel's flexible ribbon cable carries a 20-pin FPC connector. Only 8 of the 20 pins are used for the SPI interface and power; the remainder are unused/no-connect on this module. This pinout was verified directly against the physical ribbon during bring-up and is the authoritative reference for this module — it supersedes any generic ILI9341 breakout pinout found online, which can vary by manufacturer.

FPC Pin	Signal	Function
1	GND	Ground
2	VCC	Panel logic power (3.3V)
5	SDI (MOSI)	SPI data in — serial data to the display
7	DC	Data/Command select
8	SCK	SPI clock
9	CS	SPI chip-select
10	RESET	Hardware reset (active low)
12, 13	VCC	Backlight / LED power (3.3V, dual pins on this module)
20	GND	Ground (backlight return)
3, 4, 6, 11, 14–19	N/C	No connect on this 2.4" module — do not populate

⚠ WIRE BOTH VCC PAIRS AND BOTH GND PAIRS

This module breaks VCC and GND out on two separate pin pairs each (VCC: pins 2 and 12/13, GND: pins 1 and 20) — one pair feeds the display logic, the other feeds the backlight rail. Both VCC pairs must be tied to the same 3.3V rail and both GND pairs to the same ground — do not leave either pair floating, or the backlight or panel logic will not power up correctly.

7.2 Pico ↔ TFT Wiring — Final Connection Table

This is the definitive Pico-to-panel wiring for the assembled head unit, cross-referencing the FPC pinout above against the Pico's SPI0 peripheral pins.

TFT FPC Pin (Signal)	Pico Pin (GPIO)	Notes
1 / 20 — GND	GND (e.g. physical pin 3, 8, 13...)	Common ground — tie both FPC ground pins to the same Pico GND
2 / 12,13 — VCC	3V3 OUT (physical pin 36)	Common 3.3V — tie both FPC power pins to the same 3V3 OUT rail
5 — SDI (MOSI)	GP19 (physical pin 25) — SPI0 TX	SPI data line, Pico → display
7 — DC	GP16 (physical pin 21)	Data/command select, plain GPIO output

TFT FPC Pin (Signal)	Pico Pin (GPIO)	Notes
8 — SCK	GP18 (physical pin 24) — SPI0 SCK	SPI clock, Pico → display
9 — CS	GP17 (physical pin 22) — SPI0 CSn	SPI chip-select, active low
10 — RESET	GP20 (physical pin 26)	Hardware reset, plain GPIO output

7.3 Pico ↔ ESP32-CAM Emotion Relay

Pico Pin	Function	Connects To
GP1 (physical pin 2)	UART RX	ESP32-CAM emotion relay (UART1 TX, GPIO3)
GND	Ground	Common ground with ESP32-CAM

7.4 Head Wiring Schematic — Pico, TFT, and ESP32-CAM Relay

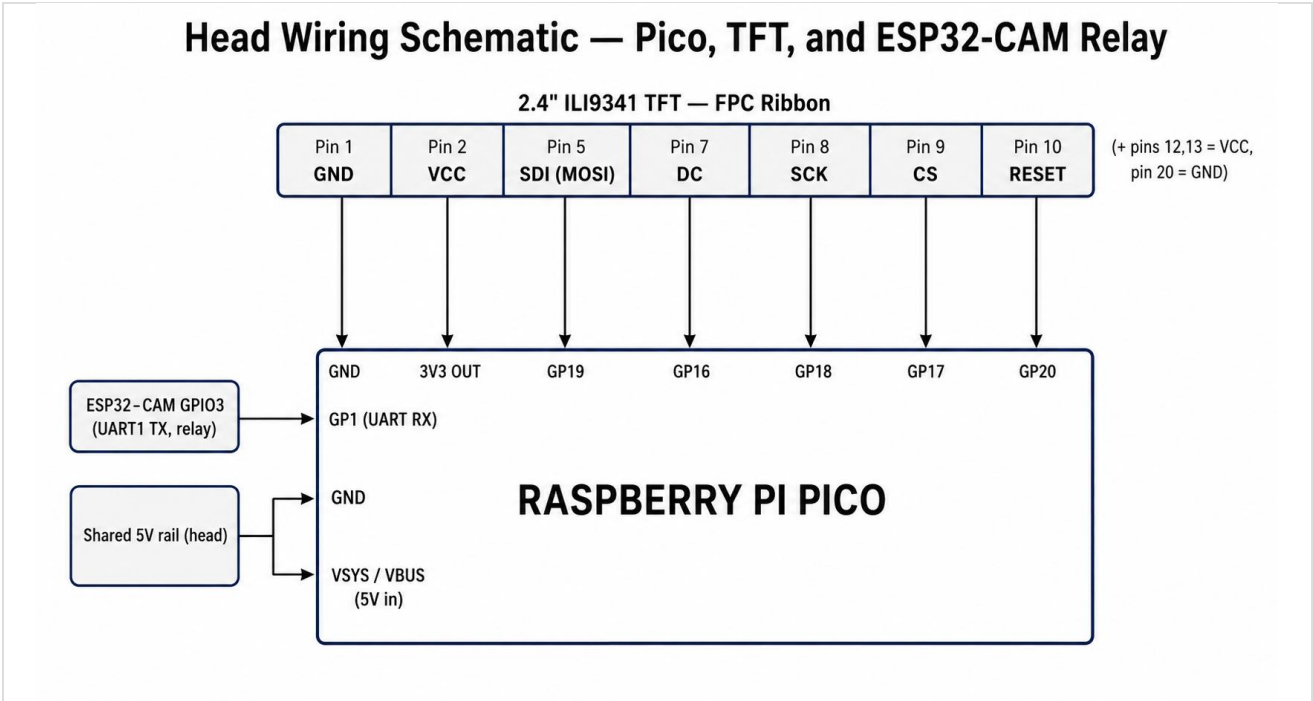


Figure 7.1 — Pico, TFT FPC ribbon, and ESP32-CAM emotion relay wiring

Only three signals cross from the ESP32-CAM into the Pico: UART1 TX (emotion text), GND, and 5V power — matching the one-directional emotion-relay design in Section 9. All SPI and reset lines run exclusively between the Pico and the TFT panel; the ESP32-CAM has no electrical connection to the display itself.

7.5 Why adam_tft.py Exists on the Pi

adam_tft.py is a Python/PIL port of the same emotion-drawing logic, intended to run on the Pi Zero 2W for headless preview, animation design, and testing without needing the physical Pico + TFT connected. It is not the live display path on an assembled robot — the Pico is. Treat adam_tft.py as a development/simulation tool: useful for iterating on new emotion animations before porting the drawing logic to the Pico's own firmware, and as living documentation of what each emotion should look like.

⚠ DO NOT WIRE THE TFT TO THE PI ON ASSEMBLED UNITS

Do not wire an ILI9341 directly to the Pi Zero 2W expecting adam_tft.py to drive the robot's actual face — on the assembled unit, the TFT is wired to and rendered by the Pico. Running

adam_tft.py standalone (python3 adam_tft.py happy) is for design/testing only.

7.6 Supported Emotions

idle · speaking · happy · sad · angry · panic · surprised · shy · sleep · thinking · reconnecting · love · confused · rizz — each has a dedicated hand-coded draw routine, driven by a shared millisecond clock for consistent animation timing. adam_tft.py's `_DRAW_MAP` dispatch table is the reference implementation these emotions should be ported from/kept in sync with on the Pico side.

7.7 Pi-Side Reference/Preview Integration Pattern (adam_tft.py)

```
from adam_tft import TFTEmotionRenderer tft = TFTEmotionRenderer() # headless preview if no
ILI9341 wired to the Pi tft.start() # preview an emotion without touching the real robot's Pico:
tft.set_emotion("happy") tft.stop()
```

8. Power Distribution & Budgeting

ADAM draws from a single battery/DC input that is stepped down and split across two physically separate zones (body and head) connected only by the 4-wire neck harness described in Section 2.3. This section documents the actual rail architecture, per-component budgets, and headroom, so future hardware changes don't silently overload a rail.

8.1 Power Architecture Overview

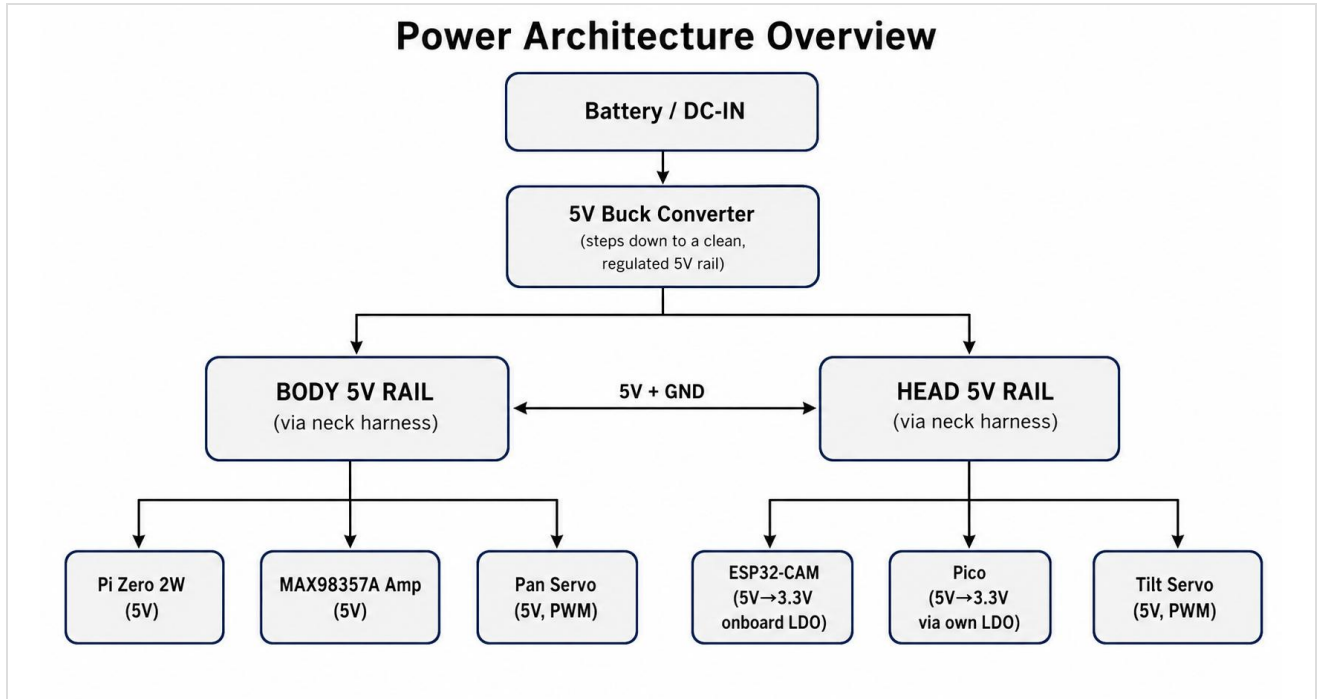


Figure 8.1 — System power rail architecture: battery to buck converter to body/head 5V rails

The Pi's own 5V pin is never used to power downstream peripherals — everything is fed from the shared buck converter output, per the original hardware design rule. This avoids brownouts on the Pi caused by servo current spikes or amplifier inrush current.

8.2 Body (Base) Power Budget

Component	Voltage	Typical Draw	Peak Draw	Notes
Raspberry Pi Zero 2W	5V	~350 mA	~700 mA	Peaks during Wi-Fi TX + CPU load (Gemini Live session)
INMP441 mic ×2	3.3V (from Pi)	~3 mA total	~5 mA	Negligible — powered from Pi's own 3V3 rail
MAX98357A amp + speaker	5V	~100 mA idle	~1.0 A	Peak during loud audio playback — dominant transient load
RCWL-0516 radar	5V	~3 mA	~3 mA	Always-on, negligible draw
MG90S pan servo	5V	~100 mA holding	~600 mA	Peak during active rotation under load
Body subtotal	5V	~556 mA	~2.31 A	Design for ≥3A continuous on the body rail

8.3 Head Power Budget

Component	Voltage	Typical Draw	Peak Draw	Notes
ESP32-CAM (AI-Thinker)	5V (onboard LDO → 3.3V)	~180 mA	~310 mA	Peaks during Wi-Fi/flash + camera capture
Raspberry Pi Pico	5V via VSYS (onboard LDO → 3.3V)	~30 mA	~90 mA	Rises while actively driving SPI + rendering at 30 FPS
ILI9341 TFT panel	3.3V (from Pico 3V3 OUT)	~80 mA	~120 mA	Backlight is the dominant draw — see Section 8.5
MG90S tilt servo	5V	~100 mA holding	~600 mA	Peak during active tilt movement
4× TTP223 touch pads	3.3V (from ESP32-CAM)	<1 mA each	<1 mA each	Negligible
Head subtotal	5V	~391 mA	~1.12 A	Design for ≥1.5A continuous on the head rail

8.4 Combined System Budget & Buck Converter Sizing

Rail	Typical Draw	Worst-Case Peak	Recommended Supply Headroom
Body 5V rail	~556 mA	~2.31 A	≥3.0 A continuous
Head 5V rail (via neck harness)	~391 mA	~1.12 A	≥1.5 A continuous
Combined (buck converter output)	~947 mA	~3.43 A (rare, non-simultaneous in practice)	5V buck converter rated ≥4.5–5.0 A continuous

⚠ SIZE THE BUCK CONVERTER TO PEAK, NOT TYPICAL, DRAW

The worst-case peak assumes the amplifier, both servos, and the ESP32-CAM Wi-Fi radio all spike simultaneously — statistically rare but possible (e.g. loud speech + rapid head turn + camera Wi-Fi retry). Undersizing the buck converter shows up as brownout resets on the Pi or ESP32-CAM exactly during these moments, which is easy to misdiagnose as a software bug. Always size the buck converter with headroom rather than to the typical-draw figure.

8.5 Neck Harness — Power Conductors

Only two of the four neck-crossing wires carry power (5V and GND); the other two are the UART2 data pair (Section 3.3 / Section 5.1). Because both the head's ESP32-CAM and Pico draw from this single 5V pair, wire gauge matters more than it would for a data-only connection.

- Use at minimum 26 AWG (ideally 24 AWG) stranded wire for the 5V and GND neck conductors to keep voltage drop low across the flexing joint
- Keep neck harness wire runs as short as the mechanical design allows — voltage drop scales with both current and length

- Route power and UART2 data pairs physically apart within the harness to reduce coupling noise into the 921600 baud UART2 link

8.6 Regulator Chain Summary

Stage	Input	Output	Regulates
Buck converter (body)	Battery / DC-IN	5V regulated	Whole-system 5V rail — feeds both body and head via neck harness
Pi Zero 2W onboard	5V	3.3V + 1.8V (internal)	Pi's own SoC, GPIO logic, I2S mics
ESP32-CAM onboard LDO	5V (from neck harness)	3.3V	ESP32 SoC, camera sensor, touch GPIOs
Pico onboard LDO	5V via VSYS (from neck harness)	3.3V	RP2040 SoC, GPIO logic
Pico 3V3 OUT pin	3.3V (post-LDO)	3.3V, $\leq 300\text{mA}$ sourced	TFT panel VCC — do not also power the ESP32-CAM's touch pads from this pin

Source: esp32cam_adam_wired_fixed.ino and the ESP32Link class in ADAM_Pi.py. Two protocol directions share UART2.

9.1 Outbound to Pi (Binary, Tag-Prefixed)

Tag Byte	Payload	Meaning
'F' (0x46)	uint32 length (LE) + JPEG bytes	Camera frame — sent at 1 FPS to match Gemini Live's cap
'T' (0x54)	4 bytes: t1 t2 t3 t4 (0/1 each)	Raw touch pad state
'G' (0x47)	1 byte gesture code	Decoded gesture: 0=none, 1=angry (slap), 2=petting, 3=stop

9.2 Inbound from Pi (Text-Line Protocol, Newline-Terminated)

Line Format	Meaning
TILT:90\n	Move tilt servo to given angle (constrained 50–120°)
EMO:happy\n	Relayed verbatim to the Pico over the UART1 relay wire — this is the live path that changes ADAM's face. Dumb relay: no validation on the ESP32-CAM side; the Pico's own firmware falls back to "idle" for unrecognized strings.

9.3 Frame Resync Logic (Pi-Side Robustness)

ADAM_Pi.py's ESP32Link._read_loop() does not trust a single byte read as a valid tag. At 921600 baud with no hardware flow control on a Pi Zero 2W, a byte occasionally mismatches by one position, and a stray JPEG byte can coincidentally equal 'F'/'T'/'G'. The reader therefore:

- Validates frame length against a sane range ($0 < \text{len} \leq 200,000$ bytes) before trusting it
- Additionally checks real JPEG markers (0xFFD8 start, 0xFFD9 end) before accepting a frame
- On any mismatch, resumes scanning byte-by-byte for the next valid tag rather than re-trusting the very next byte blindly
- Rate-limits its own resync warning prints so a desync storm doesn't itself become a CPU/IO problem
- Sleeps briefly on genuinely empty reads so the reader thread doesn't spin at 100% CPU and starve the audio threads on the 2-core Pi Zero 2W

10. Software Architecture — ADAM_Pi.py

This is the main process running on the Pi. It manages the Gemini Live session, audio pipeline, camera/gesture ingestion, servo control, TFT emotion dispatch, memory, web search, and the laptop-control tool.

10.1 Core Config

Setting	Value
Live model	gemini-3.1-flash-live-preview
Voice	Charon
Capture	plughw:0,0, S32_LE, 48kHz, stereo → downsampled to 16kHz mono for Gemini
Playback	plughw:0,0, S16_LE, 48kHz, stereo (upsampled from Gemini's 24kHz mono output)
UART port / baud	/dev/serial0 @ 921600 (env-overridable: PI_UART_PORT, PI_UART_BAUD)
Camera cadence	1 FPS, matching the ESP32-CAM's own send interval and the Gemini Live token budget
Attention timeout	30s — wake-word or motion gated
Idle nudge timeout	90s — ADAM proactively speaks if the room goes quiet

10.2 Concurrency Model

`run_session()` launches one asyncio task per subsystem, all sharing the same Gemini Live WebSocket session:

Task	Responsibility
listen	arecord subprocess → S32→S16 mono 16k conversion → mic queue
send	Streams mic queue audio to Gemini via <code>send_realtime_input</code>
receive	Reads Gemini responses: tool calls, transcripts, audio, turn-complete events
speaker	aplay subprocess; converts Gemini's 24kHz mono output to 48kHz stereo, handles end-of-turn echo draining
camera	Pulls JPEG frames from the ESP32 UART link and forwards them to Gemini at 1 FPS, gated by attention/speaking state
gesture_watch	Reacts to STOP / ANGRY / PETTING gesture codes from the ESP32-CAM — STOP interrupts speech immediately
wake_word_detector	Optional Vosk-based offline wake-word detection ('adam', 'hey adam', etc.)
idle_watcher	Proactively injects a conversational nudge after <code>IDLE_TIMEOUT_S</code> of silence
laptop_agent_healthcheck	Periodically re-verifies the laptop agent's cached IP so it self-heals in the background

10.3 Tool Declarations Exposed to Gemini

- `get_current_datetime` — local date/time
- `set_emotion` — drives the TFT face (14 emotion enum values)
- `save_memory` / `delete_memory` / `get_memory` — persistent key-value memory (`adam_memory.json`)
- `remember_person` — persistent face/person memory (`adam_faces.json`)

- `web_search` — DuckDuckGo search via the `ddgs` package, cached 30 minutes, rate-limited to one request per 5s
- `laptop_control` — dynamically built from the laptop agent's live `/actions` manifest (see Section 10)

10.4 Persistent Memory Files

File	Contents
<code>adam_memory.json</code>	Free-form key-value facts ADAM has been told to remember
<code>adam_faces.json</code>	Known people: <code>person_id</code> → {name, appearance, relationship, notes, last_seen}

11. Laptop Control Subsystem — laptop_agent.py

A new subsystem not present in the original blueprint. laptop_agent.py runs on the user's laptop (Windows/macOS/Linux) as a small Flask server, broadcasting itself on the LAN via mDNS/Zeroconf as _adam-laptop._tcp.local. so the Pi can find it without a hardcoded IP.

11.1 Why It's Modular

Every action is a small function decorated with @action(name, description, needs_value, value_hint), which self-registers into an ACTIONS dict. Two endpoints expose this:

Endpoint	Purpose
GET /actions	Self-describing manifest — the Pi builds its Gemini tool schema from this automatically
POST /control	Generic dispatcher — looks the requested action up in ACTIONS and calls it, no if/elif chain

Adding a new laptop capability (e.g. "lock_screen") requires writing one @action-decorated function and restarting the agent — nothing else in either file needs to change. The Pi's tool schema regrows automatically on its next /actions refresh (cached for 120s, force-refreshable).

11.2 Actions Currently Registered

Action	Needs Value	Description
volume_up / volume_down	No	±10% system volume
volume_set	Yes (0–100)	Set exact volume percentage
volume_mute / volume_unmute	No	Mute / unmute system audio
brightness_up / brightness_down	No	±10% screen brightness
brightness_set	Yes (0–100)	Set exact brightness percentage

11.3 Platform Backends

- Windows — pycaw (volume) + screen_brightness_control (brightness)
- macOS — osascript (volume) + brightness CLI (brightness)
- Linux — amixer (volume) + screen_brightness_control (brightness)

11.4 Discovery & Resilience on the Pi Side

- _discover_laptop_agent_ip() browses mDNS for _adam-laptop._tcp.local., caches the result for 60s, and falls back to a static LAPTOP_AGENT_IP env var if mDNS is blocked by the router
- A ConnectTimeout invalidates the cached IP immediately so the next call re-discovers rather than retrying a dead address
- A static fallback action list (_LAPTOP_ACTIONS_FALLBACK) is used only if the agent has never once answered /actions during the current run — e.g. laptop was off at Pi boot

11.5 Security Model

The Flask server binds 0.0.0.0 so the Pi can reach it over the LAN. The shared AGENT_TOKEN (set in .env) is the actual security boundary — it must be kept secret and the agent must never be port-forwarded to the public internet; it is designed for LAN-only use.

12. Serial Port Setup & Common Failure Modes

12.1 One-Time Pi Serial Setup

```
sudo usermod -a -G dialout pi sudo raspi-config → Interface Options → Serial Port "login
shell over serial?" → No "serial port hardware enabled?" → Yes grep -E
"enable_uart|dtoverlay=disable-bt" /boot/firmware/config.txt (must show BOTH enable_uart=1 and
dtoverlay=disable-bt on Pi Zero 2W — disable-bt frees /dev/serial0 from the Bluetooth UART)
sudo systemctl disable hciuart sudo reboot
```

Verify after reboot:

```
groups # must list "dialout" ls -l /dev/serial0 # symlink, group dialout, rw
for group
```

If still "Permission denied" after reboot + dialout membership, something else has the port open:

```
sudo systemctl status serial-getty@ttyAMA0.service sudo systemctl disable --now serial-
getty@ttyAMA0.service
```

12.2 Diagnostic Table — "ADAM Can't See" and Related Symptoms

Symptom	Likely Cause	Fix
Camera init FAILED / garbage frames, touch & gestures work fine	GPIO16/PSRAM conflict (see Section 6)	Confirm UART2 is on GPIO13/15, not GPIO16; verify PSRAM CS pin with a multimeter
ESP32-CAM shows "not connected" but wiring looks right	/dev/serial0 permission issue on the Pi	Run the Section 11.1 setup; disable serial-getty@ttyAMA0
UART port opens but zero data ever arrives (10s watchdog fires)	TX/RX swapped, ESP32-CAM unpowered, or baud mismatch	Verify crossover wiring; confirm both sides use the same UART2 BAUD
Frame length looks like a huge random number (millions)	Byte-level desync mid-JPEG stream	Handled automatically by the resync logic in Section 9.3 — if persistent, drop baud to 460800
Laptop volume/brightness tool calls silently fail	Laptop agent not discovered / AGENT_TOKEN mismatch	Confirm laptop_agent.py is running, same LAN, and .env tokens match on both ends

13. Setup Checklists

13.1 Python Dependencies (Pi)

```
pip install --upgrade google-genai pyaudio python-dotenv websockets pip install pyserial numpy
requests zeroconf --break-system-packages pip install webrtcvad vosk # optional,
wake-word + VAD pip install ddgs # optional, free web search pip
install luma.lcd Pillow RPi.GPIO spidev # OPTIONAL – only for adam_tft.py headless
preview/testing, # not required for the assembled
robot (Pico drives the real TFT)
```

13.2 Python Dependencies (Laptop, for laptop_agent.py)

```
pip install flask zeroconf # Windows: pip install pycaw comtypes screen-brightness-control #
macOS: brew install brightness # Linux: pip install screen-brightness-control (+ also-
utils for amixer)
```

13.3 ESP32-CAM Firmware

- Board package: esp32 by Espressif Systems (v2.0.x recommended)
- Library: ESP32Servo
- Flash over USB-serial (GPIO1/3) with the Pico relay wire physically disconnected

13.4 Pre-Wiring Checklist

- ESP32-CAM GPIO15 → Pi GPIO15 (pin 10), ESP32-CAM GPIO13 → Pi GPIO14 (pin 8) — TX/RX crossed correctly
- Touch pads: TOUCH1→GPIO12, TOUCH2→GPIO14, TOUCH3→GPIO16, TOUCH4→GPIO2
- Tilt servo → ESP32-CAM GPIO4; Pan servo → Pi GPIO12 (physical pin 32)
- Pico wired to TFT and to ESP32-CAM UART1 relay per Section 7.1 — the TFT connects to the Pico, not the Pi
- RCWL-0516 OUT → Pi GPIO16 (physical pin 36)
- PI_UART_BAUD (.env) matches UART2_BAUD (.ino) — default 921600
- AGENT_TOKEN set in both laptop_agent.py's and ADAM_Pi.py's .env files if using laptop control

13.5 Post-Flash Verification

- Serial monitor (115200 baud, USB debug UART) prints "Camera ready" and "UART2 to Pi ready"
- Pi console shows "UART port opened" and, within 10s, confirms it is receiving data (no watchdog warning)
- Touching each pad individually triggers the expected gesture in the Pi console log
- set_emotion tool call visibly changes the TFT face
- laptop_control volume_up returns {"status": "ok"} when the laptop agent is running on the same LAN

— End of Document —

DGEN Technologies Pvt. Ltd., Kolkata | Confidential Engineering Reference